

계 획 서

과제명 : Soft On-boarding Agent

팀명	VECTOR
팀장	김동준
팀원	박민준,이시영,장우석,홍재원

1. 개발과제의 개요

1. 개발과제의 개요

개발 내용 요약 : 과제목표, 내용, 기대효과 및 활용방안

1. 과제 목표

현재 IT 직군, 특히 개발팀의 가장 큰 페인 포인트는 온보딩 리소스 낭비와 신규 멤버의 초기 이탈입니다. 획일화된 온보딩 문서는 개인의 기술 숙련도를 반영하지 못해 정보 과부하를 초래하며, 시니어 개발자는 반복적인 교육으로 인해 막대한 업무 공수를 낭비하고 있습니다.

본 프로젝트는 LLM(대규모 언어 모델)과 RAG(검색 증강 생성) 기술을 활용하여, 신규 멤버의 과거 GitHub 데이터를 기반으로 개인의 기술 프로파일을 자동 분석하고 사내 코드베이스와 매핑하는 '개인화 온보딩 에이전트'를 개발하는 것을 목표로 합니다. 이를 통해 신입 구성원이 조직에 빠르게 녹아들 수 있도록 돕고, 시니어의 교육 비용을 최소화하며, 궁극적으로 조직의 지식 유통 구조를 혁신하는 DX(디지털 전환) 솔루션을 구축하고자 합니다.

2. 과제 내용

본 시스템은 신규 구성원의 GitHub 계정을 연동하여 소스코드 구조, 커밋 히스토리, PR 패턴을 분석하고, 직군별(프론트엔드, 백엔드, 데이터 엔지니어, PM/디자이너 등)로 꼭 필요한 사내 지식과 온보딩 태스크를 맞춤형으로 제공합니다. 시스템은 복잡한 프로젝트 구조를 직관적으로 파악할 수 있도록 4가지 관점의 아키텍처 (4-Way View) 대시보드를 제공합니다.

1) GitHub API 기반 기술 프로파일링 및 맞춤형 온보딩

- 신규 멤버 합류 시 GitHub 데이터를 추출하여 LLM이 개인의 기술 스택과 설계 패턴 수준을 객관적으로 진단합니다.
- 진단된 프로파일을 바탕으로 사내 코드베이스를 교차 매핑하여, 현재 수준에서 가장 빠르고 안전하게 적응할 수 있는 개인 맞춤형 온보딩 가이드를 생성합니다.
- 역할 인지형(Role-Aware) LLM 데이터 분류 파이프라인
- 프론트엔드(컴포넌트 구조/상태 관리), 백엔드(API 명세/비즈니스 로직), 데이터 엔지니어(ETL 파이프라인/로깅), PM/디자이너(이슈 맥락/기획 컨벤션) 등 직군별로 검색 컨텍스트를 격리하여 맞춤형 데이터를 자동 분류 및 구조화합니다.

2) 5개 직군을 위한 4-Way 뷰(View) 제공

- Functional View: 기능 위주로 현재 스프린트에서 개발 중인 피쳐와 에픽 현황 파악
- Data View: 데이터 구조 위주로 DB 스키마와 API 명세 흐름 파악
- Interface View: UI/UX 산출물과 실제 프론트 화면 간의 연동 및 디자인 시스템 확인
- Process & Flow View: 전체 파이프라인 진행도 및 직군 간 작업 프로세스 흐름 파악

3. 기대 효과 및 활용 방안

[기대 효과]

- 온보딩 효율 극대화 및 비용 절감: 개인 맞춤형 가이드를 통해 신입 구성원의 실무 투입 기간(Ramp-up Time)을 50% 이상 단축하고, 시니어 개발자의 온보딩 지원 공수를 주 2시간 이하로 대폭 절감합니다.
- 초기 이탈률 방어: 자신의 수준에 맞는 태스크와 가이드가 제공되므로, 초기 정보 과부하와 소외감으로 인한 90일 이내 퇴사율을 획기적으로 낮춥니다.

[활용 방안]

시니어 암묵지의 지식 자산화: LLM과 RAG 파이프라인을 통해 시니어들의 코드 컨벤션과 아키텍처 설계 의도가 지식 허브에 지속적으로 축적됩니다. 핵심 인력이 이탈하더라도 조직의 기술 자산이 사라지지 않는 영구적인 사내 지식 베이스로 활용됩니다.

2. 과제의 필요성 및 기대효과

(1) 기존 기술(과제)의 현황, 문제점 및 개선방안

1) 국내·외 연구, 산업 현황, 문제점 및 전망

IT 산업의 빠른 성장과 함께 개발 인력 수요는 꾸준히 증가하고 있지만, 신입 구성원의 조직 적응 문제는 여전히 해결되지 않은 과제로 남아 있습니다. 현재 대부분의 기업은 사내 위키 구축이나 멘토 제도를 통해 온보딩을 운영하고 있으며, Linear·Jira 같은 협업 툴에 AI 기능을 접목하는 방향으로 발전하고 있습니다. 그러나 이러한 도구들은 '업무 관리'에 특화되어 있을 뿐, 신입 구성원이 조직에 빠르게 녹아드는 문제를 직접적으로 해결하지는 못합니다. 기존 온보딩 방식에는 세 가지 명확한 한계가 있습니다.

첫째, 모든 신입에게 동일한 자료를 제공합니다. React 경험이 5년인 개발자와 첫 취업자가 같은 온보딩 문서를 받는 구조는, 숙련자에게는 시간 낭비이고 미숙련자에게는 정보 과부하입니다.

둘째, 개인의 GitHub 데이터가 전혀 활용되지 않습니다. 신입 구성원의 커밋 히스토리, PR 패턴, 코드 구조에는 해당 개인의 기술 수준을 객관적으로 파악할 수 있는 데이터가 이미 축적되어 있습니다. 그러나 현재 기업의 온보딩 프로세스는 이를 완전히 무시합니다.

셋째, 시니어 개발자의 반복 교육 비용이 큼니다. 코드 컨벤션 설명, 프로젝트 구조 안내 등 반복적인 구두 교육으로 인해 시니어 1인당 주 평균 6~8시간의 개발 공수가 온보딩 지원에 소모되고 있습니다.

이러한 문제를 배경으로, LLM과 RAG 기술의 발전은 개인의 기술 이력 데이터를 분석해 조직의 코드베이스와 자동으로 매핑하는 '개인화 온보딩 에이전트'를 현실적인 솔루션으로 만들고 있습니다. 저희는 바로 이 지점에서 본 과제를 기획했습니다.

2) 프로젝트의 필요성을 구체적으로 기술

본 과제의 핵심 아이디어는 단순합니다. 신입 구성원이 자신의 GitHub 계정을 연동하면, 시스템이 소스코드 구조·커밋 히스토리·PR 패턴을 자동으로 분석해 개인의 기술 프로파일을 생성하고, 이를 회사의 코드베이스와 교차 매핑하여 "이 사람에게 지금 꼭 필요한 온보딩"을 제공하는 것입니다.

직군에 따라 가이드의 방향도 달라집니다. 프론트엔드 개발자에게는 회사의 컴포넌트 구조와 상태 관리 방식을, 백엔드 개발자에게는 비즈니스 로직 Class와 API 명세를, 데이터 엔지니어에게는 ETL 파이프라인과 로깅 컨벤션을 중점적으로 안내합니다. PM과 디자이너의 경우 연동된 레포의 이슈·문서 맥락을 분석해 제품의 히스토리와 사내 기획·디자인 컨벤션을 전달합니다.

결국 이 솔루션이 필요한 이유는 명확합니다. 사람마다 다른 기술 배경을 무시한 획일적 온보딩은 조직과 개인 모두에게 비효율이며, 이미 GitHub에 존재하는 데이터를 활용하면 그 비효율을 구조적으로 해결할 수 있기 때문입니다.

(2) 과제개발 혹은 제작에 따른 기대효과

기술적으로는 GitHub API 기반의 개인 기술 프로파일링 엔진과 역할 인지형(Role-Aware) RAG 파이프라인 기술을 확보합니다. 단순한 언어 스택 분류를 넘어 코드 설계 패턴 수준까지 진단하고, 직군별로 검색 컨텍스트를 격리하여 꼭 필요한 사내 지식만을 선별 제공하는 것이 기술적 핵심입니다.

경제·산업적으로는 아래와 같은 효과를 기대합니다.

항목	현재	목표
신입 Ramp-up Time	평균 3개월	6주이내(50%단축)
시니어 온보딩 지원 공수	주 6~8시간	주 2시간 이하(75%절감)
90일 이내 초기 이탈률	평균 20%	8%이하
PR리젝션율(컨벤션 미준수)	평균 35%	10% 이하

무엇보다 이 솔루션의 가장 큰 가치는 온보딩 비용을 줄이는 것에 그치지 않는다는 점입니다. 시니어의 암묵지를 AI 지식 허브에 지속적으로 축적함으로써, 핵심 인력이 이탈하더라도 조직의 기술 자산이 사라지지 않는 구조를 만듭니다. 이것이 본 과제를 단순한 챗봇 개발이 아닌, 조직의 지식 유통 구조를 바꾸는 DX 솔루션으로 정의하는 이유입니다.

3. 과제목표 및 내용

1) 과제 최종목표

가. 정성적 목표:

- 신규 인력의 빠른 적응 및 업무 공백 최소화: 프로젝트를 AI가 분석해 시각화하고, 전 담당자의 작업 이탈 지점을 안내하여 도메인 학습 장벽을 낮추고 매끄러운 업무 인계를 지원.

- 팀 전체의 업무 효율성 및 몰입도 향상: 반복적인 인수인계와 기본 구조 파악에 소모되는 불필요한 커뮤니케이션 비용을 줄여, 실무에 집중할 수 있는 환경을 조성합니다.

- 맥락 기반의 직관적 협업 체계 구축: 다이어그램 내 특정 좌표를 링크하는 Q&A 시스템을 도입하여, 팀원 간 정보의 비대칭성을 해소하고 신속·정확한 의사결정 문화를 정착시킵니다.

나. 정량적 목표:

- 초기 온보딩 및 업무 인수인계 소요 기간 50% 단축 (도메인 파악부터 첫 실무 티켓 착수까지의 리드타임 기준)

- 팀 내 커뮤니케이션 소요 리소스 50% 절감 (신규 인력 교육 및 질의응답 대응 시간 기준)

- 프로젝트 마일스톤 준수율 95% 달성 (실시간 진행률 트래킹 및 지연 경고 시스템을 통한 리스크 조기 차단)

2) 과제 범위 및 주요 기능

가. 스페이스 및 권한 관리

- 관리자(PM) 권한의 '팀 스페이스' 개설 및 '팀 코드' 기반 프라이빗 초대 메커니즘 구현.

- 가입 시 직무(개발, 기획, 디자인 등) 필수 지정을 통한 사용자별 맞춤형 초기 환경 세팅.

나. AI 기반 특화 데이터 파이프라인

- GitHub 레포지토리 연동을 통한 AI 자동 파싱 및 핵심 데이터 구조 추출.

- 수동 동기화 제어: 뷰 단위 수동 업데이트 지원 및 갱신 시점별 버전 히스토리 아카이빙.

다. 직무별 맞춤형 시각화 대시보드

- Data View(데이터 관계), Functional View(시스템 모듈), Process & Flow View(프로세스), Interface View(화면 흐름) 제공.
- View 내의 컴포넌트별 버전 히스토리 제공

라. 개인 맞춤형 대시보드 (My Page)

- 개인에게 할당된 업무(컴포넌트 단위)만을 분리하여 노출.
- 진행률 슬라이더 및 상태값 변경을 지원하는 직관적인 진척도 업데이트 UI 제공.

마. 실시간 진척도 연동 및 인수인계 관리 시스템

- 업무 이탈 지점 시각화: 전임자의 최종 작업 위치(최종 수정 로그, 진행률)를 다이어그램에 하이라이트하여 끊김 없는 인수인계 지원.
- 개인 진행도 데이터를 메인 다이어그램 해당 노드에 실시간 반영
- 할당된 개발 기간 대비 실제 진행률을 분석하여, 지연 예상 시 PM 전용 대시보드에 실시간 경고 송출.

바. Q&A 시스템

- 특정 뷰 내의 컴포넌트 좌표를 직접 링크하는 Q&A 게시판 운영.
- 링크 클릭 시 해당 구조 화면으로 이동하여 명확한 커뮤니케이션 지원.

3) 비기능적 요구사항

- 성능: AI가 생성한 시각화 데이터는 DB 및 캐시 레이어에 즉시 적재하여, 중복 요청 시 재생성 과정 없이 1초 이내의 쾌적한 화면 렌더링 보장.
- 보안: GitHub 연동 시 OAuth 2.0 인증 적용 및 토큰 암호화 처리. 팀 코드 발급 체계를 통해 프로젝트 스페이스 간 데이터를 논리적으로 완벽히 격리.
- 호환성: 다양한 환경을 고려하여 PC, 태블릿, 모바일 등 멀티 디바이스에서 최적화된 해상도를 제공하는 반응형 웹구현.

4. 추진전략 및 방법

(1) 추진전략 및 방법

TDD 방법론을 통한 결함 최소화 :

프로젝트의 안정성을 확보하기 위해 코드 작성 전 테스트 케이스를 먼저 정의하는 TDD방식을 채택하여 프로젝트의 개발 요구사항을 정밀하게 검증하여 고품질의 서비스를 구현합니다.

자료수집 방안 :

1. GitHub API 기반의 프로젝트 단위 데이터 불러오기 :

GitHub REST API를 통해 프로젝트 단위의 데이터를 추출합니다.

2. LLM기반 직군별 맞춤형 데이터 분류 :

GitHub에서 불러온 프로젝트데이터를 LLM에 입력하고, 프롬프트 엔지니어링을 활용하여 개발 직군별 꼭 알아야할 핵심 데이터와 온보딩 태스크로 자동 분류 및 구조화 후 데이터베이스에 저장합니다.

환경 구축 방안:

1. Frontend : React기반으로 웹 페이지 구현.

2. Backend : Spring boot 프레임워크를 기반으로 서버 구축 및 로직 구현.

3. DB : supabase를 이용하여 직군별 분류 데이터 관리.

개발 절차:

1단계) 기획 및 테스트 케이스 설계 : 직군별 온보딩 시나리오 구체화, Github 데이터 추출 규격 정의, 실패하는 테스트 코드 작성.

2단계) Github api 및 기본 ui개발 : Github 프로젝트 연동 api 구현, 기본 웹 ui구현.

3단계) 핵심 로직 구현 : 직군별 필요한 다이어그램 및 데이터를 대시보드에 출력하는 핵심 기능 구현.

4단계) AI 모듈 추가 : 추출된 Github 데이터를 LLM에 전달하고 데이터 베이스에 저장하는 파이프라인 구축.

5단계) 최종 검증 : 실제 웹에서 사용자 정보 및 github 링크 입력시 직군별 대시보드에 데이터가 완벽하게 제공되는지 검증.

(2) 추진체계

김동준 : Functional View

이시영 : Data View

장우석 : Interface View

박민준 : Process & Flow View

홍재원 : Data Analyst & AI Engine

5. 추진일정

세부내용	수행기간(주)														비고
	1	2	3	4	5	6									
기획 및 테스트 케이스 설계							-								
Github api 연동															
기본 ui 개발															
핵심 로직 구현															
AI api 연결 및 데이터베이스 구현															
최종 검증															

6. 참고문헌

[화려한 스펙에 채용했더니 실무는 '팍'...졸업장 쏟아져도 쓸만한 신입 없다[대학 대전환]

(<https://view.asiae.co.kr/article/2025092220122753011>)

[3년을 채우지 못하는 신입들... '회사는 왜 모르는가' - CIVICNEWS(시빅 뉴스)]

(<https://www.civicnews.com/news/articleView.html?idxno=38508>)

